

Security and privacy audit — 2026-09-29

Outcome

The high-risk client relay and client-side Cloudinary signing secret have been removed from the `fixes` working tree. Firestore owner/public separation is deployed to the live `findlostpuppy` project. Additional release hardening now disables Android backup, prevents exact coordinates from entering public location text, restricts private Firebase Storage paths, and protects external tabs from opener access.

Two account/repository actions remain: confirm that the historically exposed Cloudinary API secret was rotated and review Cloudinary activity, and remove the historical tester roster/secret-bearing commits from the public Git history using a coordinated history rewrite. The current client changes still need to be released to web/Android users.

Audit performed on local `fixes` at `fa0dff8`, GitHub default-branch files, the public GitHub Pages JavaScript, and existing local Android web assets. No application code, cloud rules, credentials, or remote data were changed. No commit, push, deployment, or history rewrite was performed. Secret values are deliberately omitted from this report.

Local remediation prepared after the audit

The `fixes` working tree removes the client-side Cloudinary secret/signing code and KVDB relay; sanitizes shared Firebase report and sighting payloads; restricts profile, pet, and SAFE-report ownership; removes the tracked tester roster; disables Android backup; removes exact coordinates from shared location strings; and updates privacy disclosures. Firestore rules were deployed successfully on 2026-09-29. Firebase Storage is not enabled for this project, so the prepared Storage rules could not be deployed; Cloudinary remains the active media provider.

Findings

1. Critical: Cloudinary API secret published in source and client bundles

- `src/services/storageBucketService.ts:12` contains a literal API-secret fallback and references `VITE_CLOUDINARY_API_SECRET`.
- `src/services/storageBucketService.ts:389` uses this secret to sign media deletion requests in the browser.
- GitHub reports `jksurampudi5/findlostpuppy` as public. The default-branch version contains the secret assignment.
- A read-only fetch of the public website returned HTTP 200; its `assets/index-Ci0YYeZh.js` contains the exact hardcoded secret.
- The fresh local production build and existing `android/app/src/main/assets/public/assets/index-DDdDyBBA.js` also contain it. This verifies local Android assets, not the exact AAB currently installed

by Play testers.

- `.env.example:21` encourages configuring the same secret as a frontend variable. Moving the literal into a `VITE_` variable does not remove client exposure. Impact: if still valid, the exposed credential can authorize Cloudinary operations, including media deletion. Its validity was not tested by making privileged API calls.

Remediation: revoke/rotate the exposed credential in Cloudinary, remove all client-side secret/signing code, and implement authenticated server-side deletion with ownership checks. Rebuild and release web/Android clients. Review Cloudinary activity and usage. Clean repository history only after rotation and with explicit approval; deleting the current line cannot revoke copies already downloaded.

Cloudinary explicitly prohibits exposing API secrets in client code:
https://cloudinary.com/documentation/developer_onboarding_faq_find_credentials

2. High: automatic full-database export to an external relay

- `src/App.tsx:24` eagerly imports the admin page, which imports `cloudSyncService`.
- `src/services/cloudSyncService.ts:21` installs an online listener and a 45-second interval when its singleton is constructed. These do not require authentication or consent.
- `src/services/cloudSyncService.ts:77` targets `kvdb.io`; the request has no authorization header.
- `src/services/cloudSyncService.ts:102` exports and POSTs the local database.
- `src/services/storageService.ts:2147` includes users, owner profiles, pets, reports, sightings, listing reports, user reports, and blocked users. These records can contain personal contact/location information and moderation information.
- The same relay code exists on GitHub's default branch and in the public website bundle. Runtime evidence: on `localhost:5173`, a fresh unauthenticated browser attempted a relay GET and POST preflight after the normal 45-second interval. All external requests were blocked. A second test replaced relay fetches with in-memory responses and accelerated only the 45-second timer to one second; it captured POST payloads with all export categories above. Profiles were empty in the guest runtime test after app startup cleanup. The inclusion of populated personal records is established by the exporter and data models, not by transmitting real user data.

The real relay was not read or written, so successful production uploads, stored contents, and relay-side access restrictions remain unverified.

Remediation: remove/disable this relay path, use authenticated scoped backend synchronization, and investigate whether previous relay copies exist. Do not assume Firebase account deletion removes relay copies. Preserve appropriate incident evidence before any cleanup.

3. High: database read rules do not enforce promised contact privacy

- `firebase.firestore.rules:19`: any authenticated account can read every profile.
- `src/services/firebaseSyncService.ts:168`: owner profile payloads contain phone, email, address/street, and latitude/longitude.
- `firebase.firestore.rules:32`: missing reports are publicly readable.
- `src/services/firebaseSyncService.ts:306`: contact fields are uploaded regardless of `showPhone` / `showEmail`; flags do not redact their values.
- `firebase.firestore.rules:39`: any authenticated account can read all sightings, including reporter phone/email and coordinates uploaded by `src/services/firebaseSyncService.ts:347`.
- `src/services/firebaseSyncService.ts:397` explicitly fetches complete collections to the client.

Masking a phone number in a card does not prevent reading the underlying payload. These rules were verified in local and remote source, but the currently deployed Firebase rules were not retrieved. Live unauthorized access was not attempted.

Remediation: separate public listing fields from private contact/location data; enforce owner/admin/authorized-recipient access on the server; scope reads accordingly. Cover rules with emulator tests for unauthenticated users, unrelated users, owners, and admins.

4. High: missing ownership checks on some backend writes

- Profile create/update requires the submitted email to match the caller, but does not require the document ID to be the caller's UID. A caller could target another profile document while submitting their own email.
- Sighting creation does not require `reporterUserId` to match the authenticated UID.
- The prior Firebase Storage rules permitted broad reads. The prepared rules now make `users/`, `profiles/`, and `pets/` owner/admin-only and retain shared access only for broadcast media. The project currently has no Firebase Storage bucket, so these rules are dormant until Storage is enabled.

These are source-rule authorization defects; production exploitation was not attempted. Fix owner checks and test against the Firebase emulator before deploying.

5. Medium: privacy disclosures and diagnostics do not match implementation

- `src/data/legal/legalContent.ts:154` describes storage as local and says private home addresses are never collected. The app syncs profiles, including street/address fields, to Firebase and has the relay path described above.
- `src/utils/geolocationHelper.ts:356` and `:429` send latitude/longitude to OpenStreetMap Nominatim and BigDataCloud for reverse geocoding. IP fallbacks contact `ipwho.is` and `freeipapi.com`; postal lookup uses `api.postalpincode.in`.
- `src/utils/geolocationHelper.ts:105` logs precise coordinates and location diagnostics, including street information; the inspected call is not limited to development builds.
- `src/services/storageService.ts:96` contains a baseline report with unmasked contact information compiled into the client. Consent to publish that baseline contact data was not established by this review.

These observations do not establish malicious intent or covert camera recording. The reviewed camera component requests video only after its consent/action state, requests no audio, and stops tracks on closure. This was source inspection, not a full device-camera test.

Remediation: align notices with actual destinations and retention; remove precise production logs; review baseline-data publication consent; send only the minimum location precision each feature needs.

6. Environment files: currently excluded, historically committed

- Current `.env` is ignored and not tracked; `.env.example` is tracked.
- Public `https://jksurampudi5.github.io/findlostpuppy/.env` returned HTTP 404.
- `.env` exists in historical commits, including `4cd1ddc`; `8ba1f75` removed it on September 21. Removing a file from the latest tree leaves it in history.
- Inspected historical versions contain Firebase client configuration, Cloudinary cloud/upload configuration, and older Supabase client variables. Their presence is not automatically proof of a

privileged secret. The Cloudinary API secret in current source is the confirmed privileged-secret exposure.

Firebase client configuration is normally public; security depends on rules and appropriate API restrictions: <https://firebase.google.com/docs/projects/api-keys>

7. Tester email list published in the repository

- `testers.csv` contained seven tester email addresses in the public repository and remains recoverable from Git history.
 - The current working-tree copy has been removed. A normal future commit removes it from the latest branch but does not erase historical copies.
- Remediation: notify affected testers as appropriate, avoid storing tester rosters in source control, and include this file in the planned history cleanup after preserving any operational copy outside the public repository.

Validation and limitations

- `npm run build` : passed, including TypeScript; existing chunk-size/dynamic-import warnings.
- `npm run lint` : exit 0 with existing warnings.
- `npm audit` : three moderate findings through the development-tool chain `@capacitor/cli → xcode → uuid` ; no high/critical advisory findings. The registry's suggested fix downgrades the Capacitor CLI and was not applied because it would desynchronize the Capacitor 8.5.2 toolchain.
- Automated browser checks at `localhost:5173` used an isolated browser and blocked external browser traffic. No real user credentials or cloud mutations were used.
- Read-only GitHub metadata/file checks and public deployment asset inspection completed.
- A tracked-text pattern scan found no additional matches for the tested private-key/GitHub-token/AWS-key/Stripe/Resend/direct-credential-assignment patterns. It is a heuristic, not a clean bill of health; the Cloudinary fallback was found through separate manual review.
- Targeted `.env` history inspection completed. An exhaustive all-history filename traversal was stopped after taking too long; this is not a complete historical secret scan.
- GitHub security-alert/push-protection settings, deployed Firebase rules, Cloudinary restrictions/logs, actual relay contents, and the distributed Play Store AAB were not verified.
- Production and Android builds passed after hardening. The Firebase/privacy contract verification also passed.

Priority order

1. Rotate the exposed Cloudinary credential and review account activity. 2. Remove the external relay and move privileged media operations to an authenticated backend. 3. Correct database/storage access controls and private/public data separation, then validate with emulator security tests. 4. Release corrected clients; investigate old relay data and review disclosures/logging. 5. Complete a dedicated full-history secret scan and enable appropriate repository secret protection. Any history rewrite or publishing requires explicit approval.